Davante

7

SISTEMAS INFORMÁTICOS

Administración de sistemas operativos libres

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Arquitectura UNIX y la línea de comandos 5

2.1. La línea de comandos 5

/ 3. Administración de usuarios y grupos locales 6

3.1. Creación de usuarios 6

3.2. Administración de grupos 6

3.3. Eliminación de usuarios y grupos 7

/ 4. Usuarios y grupos predeterminados 7

4.1. Usuarios predeterminados 7

4.2. Grupos predeterminados 8

/ 5. Administración del rendimiento 8

5.1. Configuración de energía 8

5.2. Archivos de red sin conexión 9

/ 6. Administración de permisos 9

6.1. Configuración de permisos 9

6.2. Sudo y permisos elevados 10

6.3. Buenas prácticas para la seguridad 10

© MEDAC ISBN: 978-84-19885-67-8
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 7. Cuentas de servicio y automatización de tareas 10

7.1. Seguridad en las cuentas de servicio 11

7.2. Automatización de tareas 11

/ 8. Almacenamiento seguro 11

8.1. Seguridad y cifrado 12

/ 9. Auditoría de usuarios y monitorización 12

9.1. Herramientas de auditoría 12

9.2. Monitorización y generación de informes 13

/ 10. Administración avanzada de usuarios y grupos 13

10.1. Configuración de políticas de acceso 13

10.2. Uso de recursos 13

10.3. Listas de control de acceso (ACL) 14

10.4. Herramientas de gestión avanzada 14

/ 11. Copias de seguridad y mantenimiento del sistema 14

11.1. Backup y recuperación de datos 14

11.2. Actualización y mantenimiento del sistema 15

/ 12. Caso práctico 1: “Crear una carpeta genérica en Ubuntu” 15

/ 13. Caso práctico 2: “Actualización del sistema en Ubuntu” 16

/ 14. Resumen y resolución del caso práctico de la unidad 17

/ 15. Bibliografía 19

OBJETIVOS

Saber administrar usuarios y grupos en sistemas operativos libres.

Optimizar sistemas operativos para ordenadores portátiles.

Gestionar archivos de red sin conexión.

Utilizar herramientas de automatización para la administración del sistema.

/ 1. Introducción y contextualización práctica

En esta unidad estudiaremos las herramientas para la administración de sistemas operativos libres, poniendo un especial énfasis en la administración de usuarios y grupos, la optimización de sistemas para ordenadores portátiles y el manejo de archivos en red sin conexión. En un entorno empresarial, es muy importante comprender y saber utilizar adecuadamente estas herramientas para asegurar un funcionamiento correcto y seguro de los sistemas.



Fig.1. Usuarios y sus conexiones.

A lo largo de este tema, aprenderemos a crear y gestionar usuarios y grupos utilizando líneas de comandos y herramientas avanzadas; estudiaremos buenas prácticas para optimizar el rendimiento de los sistemas operativos en ordenadores portátiles, y descubriremos cómo manejar archivos de red sin conexión. Al terminar esta unidad, serás capaz de utilizar las herramientas necesarias para administrar sistemas operativos libres de manera adecuada, asegurando la estabilidad y la seguridad de los entornos informáticos que estén bajo tu responsabilidad.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Optimización de los equipos portátiles de una empresa"

https://on.soundcloud.com/74bvJBbj6A28Y7Cq8

/ 2. Arquitectura UNIX y la línea de comandos

La mayoría de los sistemas operativos libres, como Linux y BSD, están basados en UNIX, una arquitectura robusta y flexible desarrollada en los años 70. UNIX ha influido de forma importante en el diseño y funcionamiento de muchos sistemas operativos modernos, incluyendo algunos tan utilizados como macOS y varias distribuciones de Linux. Varias de sus características principales son:

- **Multitarea y multiusuario**: Permite que múltiples usuarios y procesos trabajen simultáneamente.
- **Estructura modular**: Utiliza una colección de herramientas y utilidades sencillas que pueden combinarse para realizar tareas complejas.
- **Sistema de archivos jerárquico**: Organiza archivos y directorios en una estructura de árbol, facilitando la organización y el acceso a los datos.
- **Portabilidad**: El diseño de UNIX permite su adaptación a diferentes plataformas de hardware.

2.1. La línea de comandos

Es una **interfaz** de usuario que permite a este interactuar con el sistema operativo mediante la introducción de **comandos de texto**. A diferencia de las interfaces gráficas de usuario (GUI), donde las acciones se realizan mediante clics y menús gráficos, la línea de comandos permite ejecutar comandos directamente, lo que puede ser más eficiente y potente para los administradores de sistemas y usuarios avanzados. Entre otras, algunas de las **ventajas** más interesantes de la línea de comandos son:

- **Eficiencia y rapidez**: Permite ejecutar tareas rápidamente sin la necesidad de navegar por múltiples menús.
- **Automatización**: Facilita la creación de *scripts* para automatizar tareas repetitivas, ahorrando tiempo y reduciendo errores.
- **Mayor control**: Proporciona un acceso más profundo y preciso a las configuraciones y funcionalidades del sistema operativo.
- **Uso de recursos**: Consume menos recursos del sistema en comparación con las interfaces gráficas.



Vídeo 1. "Introducción a los comandos de Linux"

https://bit.ly/30IbFHL

Para entender cómo funciona la línea de comandos, veamos algunos ejemplos básicos de comandos típicos en un sistema UNIX/Linux en esta tabla:

Comando	Acción o uso
ls	Muestra una lista de archivos y directorios en el directorio actual.
cd	Cambia el directorio actual al directorio especificado.
pwd	Muestra la ruta completa del directorio actual.
touch	Crea un archivo vacío con el nombre especificado.
mkdir	Crea un nuevo directorio con el nombre especificado.
rm	Elimina el archivo especificado.
Rmdir	Elimina el directorio vacío especificado.

Tabla 1. Lista de comandos básicos en UNIX.

/ 3. Administración de usuarios y grupos locales

Es **una de las tareas principales en administración de sistemas operativos libres**. Las operaciones principales dentro de esta tarea están formadas por la **creación, borrado y modificación de los usuarios y los grupos**.

3.1. Creación de usuarios

Estas son los principales **comandos** para crear un usuario y poder definir su contraseña:

- **adduser**: Permite crear un nuevo usuario en el sistema. Por ejemplo, para crear un usuario llamado pepito (imagen):

```
# sudo adduser pepito
```

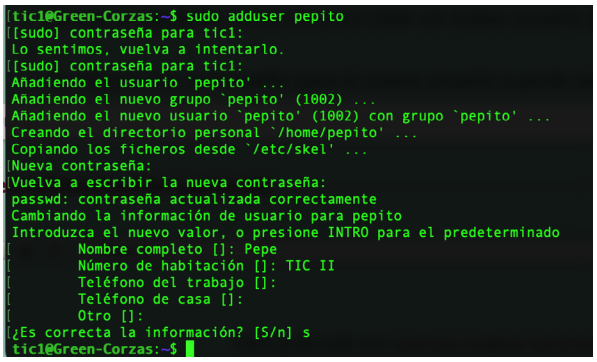


Fig.2. Ejemplo de creación de usuario.

- **passwd**: Se usa para establecer la contraseña de un usuario. Por ejemplo, para cambiar la contraseña del usuario:

```
# sudo passwd pepito
```

3.2. Administración de grupos

Algunos de los **comandos** que permiten administrar los grupos de usuarios del sistema son:

- **groupadd**: Se utiliza para crear un nuevo grupo en el sistema. Por ejemplo, para crear el grupo amigos_de_pepito:

```
# sudo addgroupamigos_de_pepito
```

- **usermod**: Para asignar un usuario a un grupo. Por ejemplo, para añadir el usuario pepito al grupo amigos_de_pepito:

```
# sudo usermod -aGamigos_de_pepito pepito
```

- **groupmod**: Se usa para modificar un grupo existente, como cambiar su nombre. Por ejemplo, para cambiar el nombre del grupo amigos_de_pepito por vecinos_de_pepito:

```
# sudo usermod -aGamigos_de_pepito pepito
```

3.3. Eliminación de usuarios y grupos

- **userdel**: Este comando se utiliza para eliminar un usuario del sistema. Por ejemplo, para eliminar el usuario pepito:

```
# sudo userdel pepito
```

- **groupdel**: Este comando se emplea para eliminar un grupo del sistema. Por ejemplo, para eliminar el grupo vecinos_de_pepito:

```
# sudo groupdelvecinos_de_pepito
```

> **¡! Sabías que...**
>
> En los sistemas operativos Linux, cada usuario y grupo tiene un identificador único conocido como UID (User ID) y GID (Group ID). El usuario root, que es el superusuario con permisos completos sobre el sistema, siempre tiene el UID 0.

/ 4. Usuarios y grupos predeterminados

En este apartado estudiaremos algunos **usuarios y grupos que existen de forma predeterminada en sistemas Linux y cumplen ciertas funciones necesarias para un correcto funcionamiento del sistema**.

4.1. Usuarios predeterminados

Existen varios usuarios predeterminados que cumplen **roles específicos** y que son importantes para el funcionamiento del sistema.

- **Root**: Es el superusuario del sistema, con permisos totales. El usuario root tiene acceso completo a todos los archivos y comandos, y puede realizar cualquier acción en el sistema, incluida la modificación del sistema operativo y la gestión de otros usuarios. Por esta razón, se recomienda usar root con precaución para evitar posibles daños al sistema.

- **Daemon**: Este usuario es utilizado por los procesos de servicio (*daemons*) que se ejecutan en segundo plano. Los *daemons* son programas que proporcionan servicios importantes como servidores web, servicios de correo, entre otros.

- **Sys**: Este usuario se emplea para tareas administrativas del sistema, como la gestión de dispositivos y el mantenimiento del sistema. Nos puede ayudar a separar funciones y mejorar la seguridad y organización del sistema operativo.

4.2. Grupos predeterminados

En Linux, los grupos predeterminados también juegan un **papel interesante en la gestión de permisos y la organización de usuarios**.

- **wheel**: Este grupo se utiliza para controlar el acceso al comando 'su', que permite a un usuario cambiar al usuario root. Los usuarios en este grupo pueden ejecutar 'su' para obtener permisos de superusuario.

- **sudo**: Este grupo permite a los usuarios ejecutar comandos con permisos de superusuario utilizando el comando sudo. Esto es preferible a usar root directamente, ya que proporciona un registro de los comandos ejecutados y puede configurarse para pedir la contraseña del usuario.

- **users**: Este es el grupo predeterminado para todos los usuarios normales. Los permisos y accesos están configurados para permitir el uso regular del sistema sin acceso a funciones críticas del sistema.

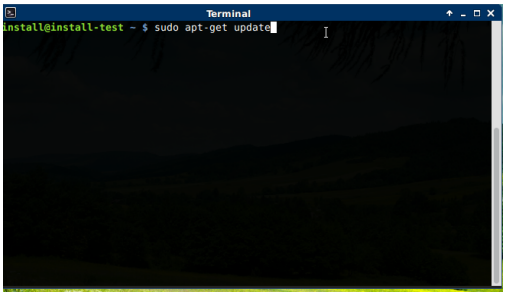


Fig.3. La instalación y actualización requiere de un usuario perteneciente a sudo para su ejecución.

/ 5. Administración del rendimiento

Para optimizar el rendimiento de los **equipos portátile**s, podemos llevar a cabo una optimización de energía y rendimiento utilizando herramientas como TLP y Powertop, y hacer uso de rsync y Unison para la sincronización de archivos de red sin conexión.

Recuerda...

La optimización de la energía no solo prolonga la vida útil de la batería de los portátiles, sino que también reduce el consumo de energía y mejora el rendimiento general del sistema.

5.1. Configuración de energía

Esta configuración nos permitirá **maximizar la duración de la batería y el correcto funcionamiento de los ordenadores portátiles**. Existen varias **herramientas en Linux** que pueden ayudar a lograr estos objetivos:

- **TLP**: Es una herramienta avanzada para la gestión de energía en Linux. Optimiza automáticamente el uso de energía del portátil mediante una serie de ajustes y configuraciones predeterminadas que se activan al instalarlo. Para instalarlo podemos ejecutar estos comandos:

```
sudo aptinstalltlptlp-rdw
sudosystemctl enable tlp
sudosystemctl start tlp
```

TLP funciona automáticamente, pero su comportamiento puede ser ajustado mediante la edición del archivo de configuración '/etc/tlp.conf'.

- **Powertop**: Es una herramienta de diagnóstico y ajuste del consumo de energía que permite identificar qué componentes del sistema están consumiendo más energía y ofrece recomendaciones para reducir el consumo. Para instalarlo y ejecutarlo, podemos hacer uso de estos comandos:

```
sudo apt install powertop
sudopowertop
```

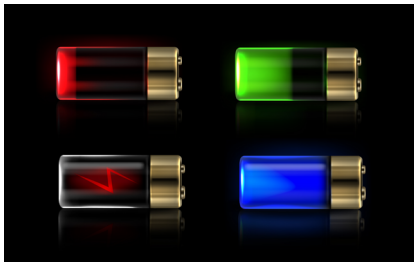


Fig.4. Representación de baterías.

5.2. Archivos de red sin conexión

La sincronización de archivos de red sin conexión nos permite asegurar que los **datos** estén **disponibles y actualizados incluso cuando no hay conexión a la red**. Dos **herramientas** muy utilizadas para este propósito son rsync y Unison:

- **rsync**: Es una herramienta muy potente y flexible para la sincronización de archivos y directorios entre ubicaciones locales y remotas.
- **Unison**: Permite la sincronización bidireccional de archivos y directorios, asegurando que los cambios en ambas ubicaciones se mantengan sincronizados.

/ 6. Administración de permisos

Esta es **otra de las principales tareas de un administrador de sistemas**. Vamos a estudiar algunos de los aspectos más importantes a tener en cuenta.

6.1. Configuración de permisos

Una correcta configuración de permisos **garantiza la seguridad del sistema**. Algunos de los **comandos** a tener en cuenta son:

- **chmod (*change mode*)**: Se utiliza para cambiar los permisos de un archivo o directorio.
- **chown (*change owner*)**: Se usa para cambiar el propietario de un archivo o directorio.
- **chgrp (*change group*)**: Se emplea para cambiar el grupo de un archivo o directorio.

6.2. Sudo y permisos elevados

El comando **sudo** se utiliza para ejecutar comandos con privilegios de superusuario sin la necesidad de cambiar al usuario root. Este método es más seguro, ya que se pueden registrar todas las actividades y se requiere la contraseña del usuario para su ejecución. Por ejemplo, para actualizar el sistema sin utilizar el usuario root, con un usuario que esté dentro del grupo sudo podemos utilizar este comando:

```
sudo apt-getupdate
```

El archivo **sudoers** define los permisos y reglas para el uso del comando sudo. Se edita utilizando el comando **visudo** para evitar errores de sintaxis.



Audio 1. "Derechos en Ubuntu"
https://bit.ly/2AM793v



6.3. Buenas prácticas para la seguridad

Con el fin de mantener la seguridad de los sistemas, es importante tener en cuenta una serie de **consideraciones importantes**:

- **Mínimos privilegios**: Utilizar permisos restrictivos por defecto (como 700) para directorios personales y permisos más abiertos (como 755) para archivos públicos. Asignar acceso sudo solo a usuarios que realmente lo necesiten, para reducir el riesgo de acceso no autorizado.
- **Revisión periódica de permisos**: Revisar de forma regular los permisos para asegurarse de que se ajustan a las políticas de seguridad.
- **Uso de grupos para gestionar permisos**: Crear grupos específicos para diferentes roles y asignar permisos basados en ellos para una gestión más segura.
- **Evitar el uso de sudo en scripts automatizados**: Siempre que sea posible, para reducir riesgos de seguridad.
- **Registrar el uso de sudo**: Revisar los registros de sudo para monitorizar las actividades realizadas con permisos de root.

/ 7. Cuentas de servicio y automatización de tareas

Las cuentas de servicio permiten **ejecutar servicios de forma aislada**, limitando el acceso de cada servicio a solo los recursos necesarios y mejorando la seguridad del sistema. Cuando las utilizamos, podemos aplicar **políticas de seguridad específicas y controles de acceso**, reduciendo el riesgo de explotación de vulnerabilidades. Además, dichas cuentas **facilitan la monitorización y la auditoría de procesos**, permitiendo identificar rápidamente cualquier actividad sospechosa o no autorizada.

7.1. Seguridad en las cuentas de servicio

- **Uso de contraseñas seguras**: Debemos asegurarnos de que las cuentas de servicio utilicen contraseñas fuertes y únicas, y debemos cambiarlas regularmente.
- **Asignación de permisos mínimos**: Hay que configurar las cuentas de servicio con los permisos mínimos necesarios para realizar sus tareas, siguiendo el principio de mínimo privilegio.
- **Auditoría y monitorización**: Es recomendable implementar sistemas de auditoría y monitorización para registrar las actividades de las cuentas de servicio y detectar posibles anomalías.
- **Segregación de servicios**: Se debe evitar la utilización de una sola cuenta de servicio para múltiples servicios, asegurando que cada cuenta esté asociada a un único servicio específico.

7.2. Automatización de tareas

La automatización de tareas de administración es útil **para mejorar la eficiencia y reducir el error humano** en la gestión de sistemas operativos. Podemos emplear **herramientas como cron y systemdtimers para programar y ejecutar tareas automáticas en Linux**:

- **Cron**: Es un demonio de Unix que permite programar tareas para que se ejecuten automáticamente en horas específicas. Los trabajos de cron se definen en archivos llamados 'crontabs'.
- **systemdtimers**: Son una alternativa moderna a cron. Utilizan el sistema de initsystemd para programar tareas. Los temporizadores de systemd aportan una mayor flexibilidad y más control sobre la ejecución de tareas programadas.

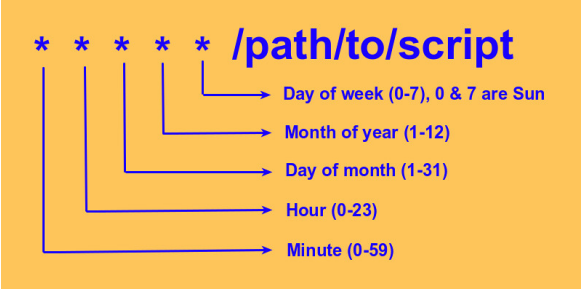


Fig.5. Sintaxis básica de un cron.

8. Almacenamiento seguro

En este apartado estudiaremos la **gestión segura del almacenamiento y de las particiones de un equipo**, muy útil para organizar y optimizar el uso del espacio en disco en los sistemas operativos libres. Existen varias **herramientas** que facilitan esta tarea:

- **GParted**: Es una herramienta gráfica para la gestión de particiones. Permite crear, eliminar, redimensionar, mover y copiar particiones sin pérdida de datos.
- **fdisk**: Es una herramienta de línea de comandos para la gestión de particiones en discos duros.
- **lsblk**: Es una herramienta de línea de comandos que muestra información sobre todos los dispositivos de bloques.

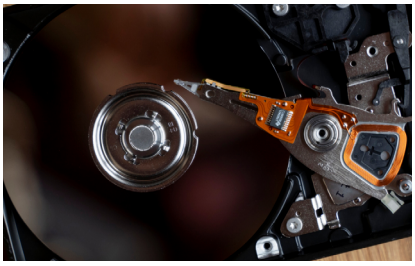


Fig.6. Disco duro.

8.1. Seguridad y cifrado

Para proteger la información sensible en portátiles, debemos implementar **técnicas de cifrado en los discos duros**. Dos herramientas muy utilizadas para este propósito son LUKS y eCryptfs:

- **LUKS**: Es una especificación estándar para el cifrado de discos en Linux que proporciona una interfaz fácil de usar para la gestión de discos cifrados.
- **eCryptfs**: Es un sistema de archivos de cifrado de pila que se puede utilizar para cifrar directorios individuales, como el directorio 'home' del usuario.

A la hora de utilizar estas herramientas, es recomendable llevar a cabo estas **prácticas**:

- **Uso de contraseñas fuertes y únicas**: Asegurarse de que las contraseñas utilizadas para el cifrado sean fuertes y diferentes de otras contraseñas del sistema.
- **Respaldo de claves de cifrado**: Mantener copias de seguridad de las claves de cifrado en un lugar seguro para evitar la pérdida de datos en caso de problemas.
- **Actualización regular del *software***: Mantener el software de cifrado y el sistema operativo actualizados para protegerse contra vulnerabilidades.

/ 9. Auditoría de usuarios y monitorización

Nos permiten mantener la **seguridad en los sistemas operativos libres**. Estas prácticas nos hacen posible rastrear las actividades de los usuarios, detectar comportamientos sospechosos y generar informes para el análisis y la toma de decisiones.



Fig.7. Representación de una auditoría.

9.1. Herramientas de auditoría

Dos de las herramientas más utilizadas para auditoría son auditd y ausearch:

- **auditd**: Es el demonio de auditoría de Linux que **recopila y registra eventos de seguridad**. Esta herramienta es importante para realizar auditorías minuciosas de las actividades del sistema. El archivo de configuración de auditd se encuentra en /etc/audit/auditd.conf. Además, se pueden definir reglas de auditoría en la ubicación /etc/audit/audit.rules.
- **ausearch**: Es una herramienta que se utiliza para **buscar y analizar los registros de auditoría generados por auditd**.

9.2. Monitorización y generación de informes

La **monitorización** de las actividades de los usuarios permite a los administradores **detectar y responder a actividades sospechosas o no autorizadas en tiempo real**. **Generar informes** periódicos facilita la **revisión y análisis de estos datos**. Algunas **herramientas** para este propósito son:

- **auditd**: Permite monitorizar los intentos de acceso a un archivo en tiempo real. Por ejemplo, podemos utilizar el siguiente comando (audita todos los tipos de acceso [r, w, x, a] al archivo especificado y los etiqueta con la clave file_access):

```
sudoauditctl -w /path/to/file -p rwxa -k file_access
```

- **aureport**: Es una herramienta para generar informes a partir de los registros de auditoría. Por ejemplo:

  » Generar un informe resumen de eventos de auditoría:

```
sudo aureport -summary
```

  » Generar un informe detallado de un tipo específico de eventos:

```
sudo aureport -x --summary
```

/ 10. Administración avanzada de usuarios y grupos

Abarca desde la configuración de políticas de seguridad para garantizar el acceso adecuado hasta el uso eficiente de los recursos del sistema.

10.1. Configuración de políticas de acceso

Las políticas de acceso a recursos establecen de forma inequívoca **quién puede acceder a qué recursos**. Se pueden utilizar grupos para gestionar permisos, asignando usuarios a grupos con roles específicos. Además, se deben definir políticas de contraseñas seguras, con requisitos de longitud mínima y complejidad.



Fig.8. Protección de directorios.

10.2. Uso de recursos

Se pueden limitar los recursos que los usuarios y/o grupos pueden utilizar, como el espacio en disco y el uso de CPU. También se puede llevar a cabo una implementación de cuotas de disco para evitar que un solo usuario consuma todo el espacio disponible.

10.3. Listas de control de acceso (ACL)

Proporcionan un mecanismo más granular para definir permisos de acceso a archivos y directorios, permitiendo asignar **permisos específicos** a múltiples usuarios y grupos. Este enfoque nos da una serie de beneficios:

- **Mayor flexibilidad** en la asignación de permisos.
- Capacidad para definir **permisos detallados** para diferentes usuarios y grupos sobre un mismo recurso.
- **Facilita** la **gestión** de acceso en **entornos con múltiples usuarios** y necesidades de permisos variadas.

10.4. Herramientas de gestión avanzada

Para gestionar usuarios y grupos de manera eficiente en entornos complejos, las herramientas de automatización como Ansible y Puppet son muy útiles. **Permiten la configuración y gestión de sistemas de forma escalable y flexible**:

- **Ansible**: Es una herramienta de automatización que utiliza archivos de configuración YAML llamados 'playbooks' para definir tareas. Es fácil de aprender, sin necesidad de instalar agentes en los nodos gestionados, y resulta adecuada para tareas de configuración y despliegue rápido.
- **Puppet**: Es una herramienta de automatización que emplea un lenguaje de configuración declarativo para definir el estado deseado de la infraestructura. Es potente y escalable, adecuada para la gestión continua de entornos complejos, además permite la configuración detallada y específica de los sistemas.

/ 11. Copias de seguridad y mantenimiento del sistema

En esta sección estudiaremos diferentes herramientas de copia de seguridad, como rsnapshot y Deja Dup, así como técnicas para la actualización y el mantenimiento del sistema.

11.1. Backup y recuperación de datos

Es importante realizar **copias de seguridad** de forma regular y configurar **procedimientos de recuperación de datos** para proteger la información y garantizar la continuidad del sistema en caso de fallos del mismo o pérdida de datos. Algunas de las **herramientas** más utilizadas para este propósito son:

- **rsnapshot**: Es una herramienta de copia de seguridad que utiliza rsync para crear imágenes instantáneas del sistema de archivos. Se puede configurar a través del archivo /etc/rsnapshot.conf para definir las rutas de los directorios a respaldar y el destino de las copias.
- **Deja Dup**: Es una herramienta de copia de seguridad fácil de usar con una interfaz gráfica, ideal para usuarios menos avanzados. Se puede configurar de forma sencilla a través de su interfaz gráfica, seleccionando los directorios a respaldar y configurando el destino de las copias (puede ser una unidad local, red o almacenamiento en la nube).

> **Investigamos...**
>
> Investiga qué son las estrategias de backup incremental y diferencial. Puedes buscar artículos académicos y en Google Scholar utilizando las palabras clave 'incremental backup' y 'differential backup'.

11.2. Actualización y mantenimiento del sistema

Mantener el sistema actualizado permite asegurar su estabilidad, seguridad y rendimiento. Utilizar las herramientas adecuadas para la gestión de paquetes y actualizaciones es una práctica muy requerida en la administración de sistemas. **El sistema de actualización depende de la distribución utilizada**:

- **apt-get** (Debian, Ubuntu y derivados): Por ejemplo, para actualizar la lista de paquetes:

```
sudo apt-getupdate
```

- **dnf** (Fedora, RHEL y derivados): Por ejemplo, para instalar un nuevo paquete:

```
sudo dnfinstallnombre_paquete
```

- **zypper** (openSUSE y SUSE Linux Enterprise): Por ejemplo, para actualizar los paquetes instalados:

```
sudo zypperupdate
```

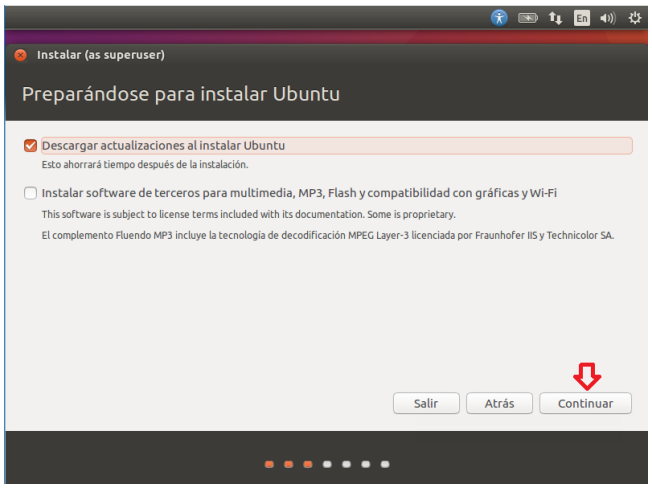


Fig.9. Descarga de actualizaciones en una distribución Debian como Ubuntu.

/ 12. Caso práctico 1: "Crear una carpeta genérica en Ubuntu"

Planteamiento: En la empresa en la que trabaja Sergio, uno de los ordenadores con Ubuntu es utilizado por varias personas a lo largo del día.

Los usuarios de la computadora desean tener una carpeta a la que puedan acceder todos los usuarios, de modo que puedan añadir y editar los archivos.

Nudo: ¿Dónde y cómo puede crear Sergio una carpeta que esté accesible a todos los usuarios y tenga permisos para añadir y editar archivos?

Desenlace: La carpeta debe ser creada por un usuario Administrador en /home. También debe abrirse el explorador de archivos de Ubuntu con un usuario Administrador y usar esta vía. Para ello, habría que ir al terminal de comandos de Ubuntu e introducir 'sudo nautilus' y la contraseña de administrador.

```
enrique@enrique-Virtual-Machine:/home$ sudo nautilus
[sudo] contraseña para enrique:
```

Fig.10. Comando sudo nautilus para abrir el explorador de archivos de Ubuntu como administrador.

Una vez abierto el explorador de archivos, hay que ir a /home y hacer clic con el botón derecho del ratón, seleccionar Carpeta nueva y definir un nombre para la nueva carpeta.

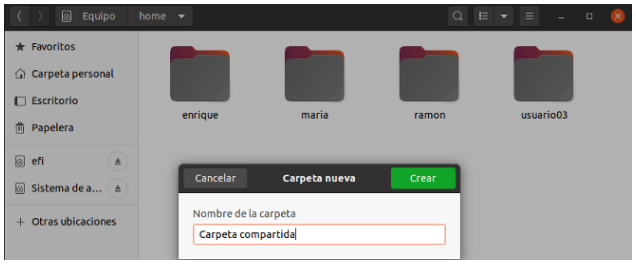


Fig.11. Crear carpeta en /home.

Por último, habrá que ir a las Propiedades de la carpeta y a la pestaña Permisos. Todos los permisos deben habilitarse como Crear y eliminar archivos. De esta forma, todos los usuarios tendrán permisos de acceso y edición a dicha carpeta.



Fig.12. Asignar permisos a una carpeta.

/ 13. Caso práctico 2: "Actualización del sistema en Ubuntu"

Planteamiento: Juan es el administrador de sistemas de una pequeña empresa que utiliza Ubuntu como sistema operativo en todos sus equipos. Ha recibido un aviso de que hay varias actualizaciones importantes disponibles, incluyendo parches de seguridad críticos. Juan debe actualizar todos los sistemas para asegurar que estén protegidos y funcionen de manera óptima.

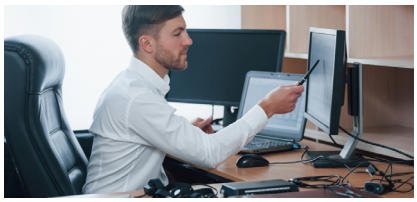


Fig.13. Juan trabajando en el proceso de actualización.

Nudo: Juan se enfrenta a la tarea de actualizar todos los sistemas Ubuntu de la empresa. Debe asegurarse de que las actualizaciones se realicen sin problemas y que los sistemas permanezcan estables durante y después del proceso. Para ello, necesita:

- Verificar y actualizar la lista de paquetes disponibles.
- Actualizar los paquetes instalados.
- Limpiar los paquetes que ya no son necesarios.
- Documentar el proceso para futuras tareas.

Desenlace:

1. Juan debe ejecutar el comando adecuado para actualizar la lista de paquetes:

```
sudo apt-getupdate
```

2. Después de actualizar la lista de paquetes, debe proceder a actualizar todos los paquetes instalados a sus versiones más recientes:

```
sudo apt-getupgrade
```

3. También puede utilizar el comando dist-upgrade para realizar una actualización más completa que maneje dependencias y posibles cambios en las versiones de paquetes:

```
sudo apt-getdist-upgrade
```

4. Después de la actualización, es una buena práctica limpiar los paquetes obsoletos o innecesarios para liberar espacio en el sistema:

```
sudo apt-getautoremove
```

5. Finalmente, Juan debe documentar todo el proceso de actualización, incluyendo cualquier problema encontrado y cómo lo resolvió, para que esta información pueda ser utilizada como referencia en el futuro.

/ 14. Resumen y resolución del caso práctico de la unidad

En este tema hemos estudiado los **tipos de software**, la **gestión de licencias** y los **procedimientos de instalación y mantenimiento** del software. También hemos destacado la importancia de cumplir con los **requisitos mínimos y recomendados para asegurar el buen funcionamiento del software**, y se ha detallado cómo realizar **instalaciones desatendidas** para ahorrar tiempo y reducir errores y costes.

Por otro lado, se ha puesto de manifiesto la importancia de mantener los **sistemas y aplicaciones actualizados para garantizar la seguridad y el rendimiento**, haciendo énfasis en la necesidad de tener una buena **documentación** y establecer **protocolos** claros. Para terminar, se han revisado las principales **herramientas ofimáticas y de Internet,** a través de una guía para seleccionar y utilizar las mismas de manera ideal.

A continuación podrás ver el esquema resumen del tema.

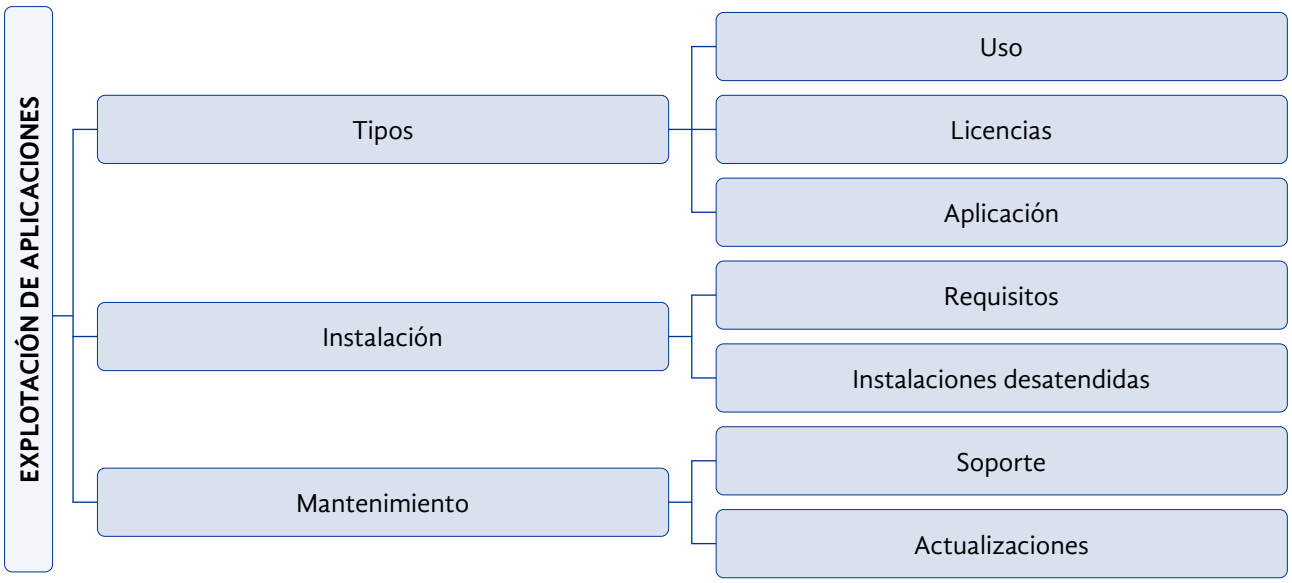


Fig.14. Esquema resumen del tema.

Resolución del caso práctico de la unidad

María, como administradora de sistemas, podría llevar a cabo las siguientes acciones:

1. **Gestión de usuarios y grupos locales**:

   » **Crear nuevos usuarios**:

   ```
   sudo useradd -m nombre_usuario
   ```

   » **Modificar usuarios existentes**:

   ```
   sudo usermod -aG grupo nombre_usuario
   ```

   » **Crear nuevos grupos**:

   ```
   sudo groupaddnombre_grupo
   ```

2. **Optimización de sistemas operativos para portátiles**:

   » **Instalar y configurar TLP para la gestión de energía**:

   ```
   sudo apt install tlptlp-rdw
   sudosystemctl enable tlp
   sudosystemctl start tlp
   ```

» **Utilizar Powertop para identificar y ajustar el consumo de energía**:

```
sudo apt install powertop
sudopowertop
```

3. Sincronización de archivos de red sin conexión:

» **Configurar rsync para la sincronización de archivos**:

```
rsync -avz /ruta/origen/ usuario@servidor:/ruta/destino/
```

» **Utilizar Unison para la sincronización bidireccional**:

```
sudo aptinstallunison
unison /ruta/origen/ ssh://usuario@servidor//ruta/destino/
```

/ 15. Bibliografía

Blum, R. (2020). *Linux Command Line and Shell Scripting Bible*. Wiley.

Galvin, P., Gagne, G. y Silberschatz, A. (2014). *Operating System Concepts* (9.ª ed.). John Wiley & Sons. ISBN: 978-1-118-06333-0.

Nemeth, E., Snyder, G., Hein, T. R., Whaley, B. y Mackin, D. (2017). *UNIX and Linux System Administration Handbook*. Prentice Hall.

Sobell, M. G. (2018). *A practical guide to Linux commands, editors, and shell programming*. Prentice Hall.

Sommerville, I. (2015). *Software Engineering* (10.ª ed.). Pearson. ISBN: 978-0-13-394303-0.

Stallings, W. (2017). *Computer Organization and Architecture: Designing for Performance* (10.ª ed.). Pearson. ISBN: 978-0-13-410161-3.

Tanenbaum, A. S. y Bos, H. (2015). *Modern Operating Systems* (4.ª ed.). Pearson. ISBN: 978-0-13-359162-0.

Purdy, K. (2020). *Linux Bible*. Wiley.

Ward, B., Smith, B. D. y Hoksbergen, M. (2015). *How Linux Works: What Every Superuser Should Know*. No Starch Press.